PampelMuse

Sebastian Kamann

July 23, 2014

Abstract

PampelMuse is a software package with the purpose of facilitating the analysis of integral-field spectroscopic observations of crowded stellar fields. It provides several subroutines to perform the individual steps of the data analysis.

1 Getting started

1.1 Requirements

PampelMuse is written in *python*. A recent version of *python* should be available on the system, *python v2.7* is recommended. In addition, several *python* packages are required to run PampelMuse:

- *numpy*[1], tested for version 1.6.1
- *scipy*[2], tested for versions ≥0.9, a version ≥0.10 is recommended because a faster algorithm for sparse matrix inversion was introduced in v0.10
- *astropy*[3], tested for version 0.2.5

Note that the versions of the packages listed above are just a recommendation. The code may also run with earlier versions. In addition to these packages, it is highly recommended that the following packages are available on the system:

- *matplotlib*[4], tested for version 1.1.1
- *PyQt4*[5]

These packages are required in order to work with the graphical user interface (GUI) that is provided with PampelMuse. When starting PampelMuse the availability of *matplotlib* and *PyQt4* is checked. If the packages are not available, the subroutines that implement the GUI will be disabled. Finally, the packages listed below are not required to run the core packages of PampelMuse, but some auxiliary routines depend on them:

- *astLib*[6], tested for version 0.7.0

[1] `http://www.numpy.org/`
[2] `http://scipy.org/`
[3] `http://www.astropy.org/`
[4] `http://matplotlib.org/`
[5] `http://www.riverbankcomputing.co.uk/software/pyqt/intro`
[6] `http://astlib.sourceforge.net/`

- *pyraf*[7]

1.2 Download and Installation

There are currently two possibilities to obtain PampelMuse that will be described in the following.

1.2.1 Using the tarball

A zipped version of the code is available at `http://path.to/pampelmuse.tar.gz`. After the download is completed, change to the folder containing the tarball and execute the following commands

```
# tar -xvzf pampelmuse.tar.gz
# cd pampelmuse/
# python setup.py install
```

This will install PampelMuse to your default *python* path. Note that the last step usually requires root privileges. If you do not have those privileges, you can define a custom path using the `--prefix` or `--home` arguments. You only have to make sure then that the path you are using to install PampelMuse is included in your `PYTHONPATH`. For example, the following commands will install PampelMuse to folders `$HOME/lib` and `$HOME/bin`.

```
# python setup.py install --home=~
# export PYTHONPATH=$PYTHONPATH:$HOME/lib/python ## for a bash-shell or
# setenv PYTHONPATH $PYTHONPATH:$HOME/lib/python ## for a c-shell
```

You probably want to store the modification of your `PYTHONPATH` variable in your shell configuration file (`.bashrc` or `.cshrc`), otherwise you have to set the variable again every time you open a new shell. For further informations on custom installation paths, check the *python* documentation.[8]

Afterwards, you should be able to execute

```
# PampelMuse --help
```

If no error message is returned, you have installed `PampelMuse` successfully!

1.3 Using the SVN repository

The maintenance of `PampelMuse` is handled via an SVN repository located at `https://subversion.assembla.com/svn/kamann-code.2`. The advantage of using the repository is that it allows you to always use the most recent version of the code. However, the disadvantage is that when updating the code I sometimes commit revisions that are not fully operational. So when updating to a new revision, be sure to check the most recent log entries.

To install from the SVN repository, first cd into the folder were you want to install the code and check out the repository:

```
# svn checkout https://subversion.assembla.com/svn/kamann-code.2/trunk
```

This should download the most recent version of `PampelMuse`. To use the code afterwards, the following modifications should be added to your shell configuration:

```
# ## for a bash shell:
# export pampelmuse_path=<path to PampelMuse> ## folder were SVN checkout was
```

[7] `http://www.stsci.edu/institute/software_hardware/pyraf`
[8] `http://docs.python.org/2/install/#alternate-installation`

Type	required?	EXTNAME
Data	y	DATA
Uncertainties	(y)	ERROR, SIGMA
Variances	(y)	STAT
Bad pixel mask	n	MASK

Table 1: Extension names used to identify the input required for the analysis in a combined FITS file. Note that either uncertainties or variances must be provided.

performed

```
# export PYTHONPATH=$PYTHONPATH:$pampelmuse_path
# export PATH=$PATH:$pampelmuse_path/pampelmuse/bin
```

Again, calling

```
# PampelMuse --help
```

should return no error message.

2 The input data

2.1 Preparing the data

First note that PampelMuse is *not* a data reduction pipeline. All analysis steps assume that the IFS data has been properly reduced and that all the instrumental artefacts have been removed. In general, the IFS data can come in two flavours:

1. As a three-dimensional data cube with two spatial axes (NAXIS1 and NAXIS2) and a spectral axis (NAXIS3). This format can always be used, even if the data were obtained with an instrument that is unknown to PampelMuse.

2. As a two-dimensional "image" with one spatial and one spectral axis. The advantage of this format is that it can also be used with instruments that have a ireegular (i.e. non-rectangular) spatial sampling. However, for this format to work PampelMuse must know how to arrange to different spaxels in the field of view. Currently this format is only supported for PMAS and ARGUS data.

Besides the actual data, PampelMuse also requires an estimate of the uncertainties or the variances for the analysis. Optionally, a bad pixel mask can be used. Data, uncertainties, and mask can either be provided as different extensions of a single FITS file or as three individual FITS files. In the former case, the routine will use the extension names to identify what is stored in which extension, see Table 1. In the latter case, the individual filenames should have the same prefix and be suffixed .dat.fits (the data), .err.fits (the uncertainties), and .msk.fits (the bad pixel mask).

If provided, the bad pixel mask should be zero for valid pixels and one for pixels that should be excluded. Instead of providing a dedicated mask, one can also set the invalid pixels to NaN and PampelMuse will ignore them in the analysis.

2.2 The reference catalogue

As `PampelMuse` does not currently include a dedicated source detection algorithm, the analysis relies on the presence of a reference catalogue, containing coordinates and magnitudes of the stars in and around the observed field. The best sources for such a catalogue are imaging data obtained with the *Hubble* space telescope (HST) or ground based adaptive optics images. Note that the quality of the reference catalogue (especially its completeness and astrometric accuracy) will strongly influence the results of the analysis. For example, a common shortcoming of catalogues generated from HST data is that the brightest stars are often missing because they were saturated in the exposures. If this is the case, try to add them manually.

The reference catalogue should be present as a plain ascii-file with the first (non-commented) line containing the column names. `PampelMuse` will recognize column names such as `ra`, `dec`, or `id` and process the column content accordingly. The reference catalogue must contain the following columns:

1. A unique identifer for every source; must be a non-negative integer
2. Two columns containing the source coordinates in a reference system. Both pixel and world coordinates are accepted, and priority is given to the latter if both are present. If provided, the world coordinates should be in units of decimal degrees.
3. A magnitude for every source in a passbband that covers at least some part of the spectral range of the IFS data.

Here is an example of a valid reference catalogue:

```
## Example reference catalogue
## Random Cluster NGC9999, generated from HST/WFPC2 data
id    ra    dec   f606w
 1  12.764311  -26.986152  13.29
 2  12.751325  -26.986431  14.87
 4  12.760123  -26.976514  12.25
 ...
```

3 The analysis with PampelMuse

3.1 INITFIT: Selecting the sources

The first step of the analysis is to identify those sources from the reference catalogue for which it is feasible to extract spectra from the IFS data. As the reference catalogue ideally was obtained from higher resolution data and typically covers a larger area on the sky, it would not make much sense to try and extract a spectrum for each star in the catalogue. Instead, we use INITFIT to make a selection. The syntax for calling this routine is

```
# PampelMuse <config> <IFS data prefix> INITFIT <reference catalogue> .
```

Note that the IFS data should be provided without the `.fits` or `.dat.fits` suffix. `config` refers to the user setup. This is explained in detail in Sect. 4.

INITFIT performs the following steps in order to select a useful subset of the available sources.

1. Opening the IFS data, checking their validity.

2. Opening the reference catalogue file, checking its validity.

3. Creating an image from the IFS data in the passband in which the magnitudes in the reference catalogue were provided.

4. Creating a mock image from the sources in the reference catalogue, again in the passband in which the magnitudes were provided. The mock image is prepared using the same pixelsize as the IFS data and the initial guess of the PSF for the IFS data (see parameters `PSFTYPE` and `PSFPARS`, Sect. 4).

5. Cross-correlating the two images just created to find the most likely location of the IFS field of view with respect to the reference catalogue. Note that this step requires that both images indeed have the same pixel scale and orientation. Hence, a minimum of information about the sampling and rotation angle must be available in the header of the IFS data. Otherwise this step is skipped.

4 The configuration parameters